

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 1 Repaso de SQL

Alumno: Tomás Rodeghiero
Año: 2026

Índice

1. Introducción	2
2. Ejercicio 1: Definición de las bases de datos	2
2.1. Inciso a) Clientes, productos, ítems y facturas	2
2.2. Inciso b) Vehículos, personas, parquímetros y estacionamiento	4
2.3. Inciso c) Clientes, automóviles, talleres y accidentes	7
3. Ejercicio 2: Carga de datos	10
3.1. Inciso a) – MySQL/PostgreSQL	10
3.2. Inciso b) – MySQL	10
3.3. Inciso b) – Oracle	11
3.4. Inciso c) – PostgreSQL	12
4. Ejercicio 3: Consultas SQL y álgebra relacional	12
4.1. a) Clientes sin ventas (base del inciso 1.a)	12
4.2. b) Vehículos que usaron el parquímetro 9 (base del inciso 1.b)	13
4.3. c) Clientes con más de 3 accidentes (base del inciso 1.c)	13
4.4. d) Máximo y mínimo monto de factura por cliente (base del inciso 1.a)	14
4.5. e) Tres consultas en álgebra relacional	14
5. Conclusión	15

1. Introducción

Este trabajo práctico funciona como un repaso integral de SQL y consolida las herramientas que se utilizarán a lo largo de la materia. La consigna se divide en tres ejercicios: primero, definir el esquema de tres bases de datos identificando claves primarias, claves secundarias, claves foráneas, integridad referencial y restricciones de dominio; en segundo lugar, cargar datos coherentes en esas bases; y por último, resolver una serie de consultas SQL y proponer otras adicionales utilizando álgebra relacional.

La resolución se acompaña de los motores de bases de datos pedidos por la cátedra (MySQL 8 o superior, PostgreSQL 14 o superior y Oracle XE 10g/11g). En cada inciso se aclara qué motores corresponden y, cuando alguno de ellos no soporta una construcción del SQL estándar (como ocurre con `CREATE DOMAIN` en MySQL u Oracle), se documenta la alternativa empleada para emularla.

A lo largo de la entrega se trabaja con tres bases distintas, una por cada inciso del Ejercicio 1. Para evitar colisiones de nombres entre tablas (varias tablas se llaman `cliente`), cada base se aísla en un esquema o base de datos propio: `practico1a`, `practico1b` y `practico1c`. Los datos cargados en el Ejercicio 2 están pensados específicamente para que las consultas del Ejercicio 3 devuelvan resultados significativos.

2. Ejercicio 1: Definición de las bases de datos

El primer ejercicio se centra en el DDL: definir las tablas con `CREATE TABLE` y declarar las restricciones que el enunciado pide explícita o implícitamente. Las cláusulas más relevantes son `PRIMARY KEY`, `UNIQUE`, `CHECK` y `FOREIGN KEY`, mientras que la noción de dominio se expresa con `CREATE DOMAIN` cuando el motor lo soporta y, en caso contrario, se emula con `CHECK` o tipos enumerados.

2.1. Inciso a) Clientes, productos, ítems y facturas

Decisiones de modelado

- `cliente.nro_cliente`, `producto.cod_producto` y `factura.nro_factura` son las claves primarias naturales del enunciado.
- Se agrega `UNIQUE` sobre `cliente.telefono` como clave secundaria razonable, ya que en este dominio no es esperable que dos clientes distintos compartan teléfono.
- `factura.nro_cliente` se declara como clave foránea hacia `cliente`, cerrando la relación 1:N entre clientes y facturas.
- `item_factura` se modela con clave primaria compuesta (`cod_producto`, `nro_factura`): cada par (producto, factura) aparece una sola vez.
- El enunciado pide que al borrar un cliente se elimine toda su información de facturas. Para cumplirlo, `factura.nro_cliente` se define con `ON DELETE CASCADE` y, en consecuencia, `item_factura.nro_factura` también se propaga con `ON DELETE CASCADE` para evitar ítems huérfanos.
- En cambio, no se debe permitir borrar un producto que ya fue facturado: `item_factura.cod_producto` se declara con `ON DELETE RESTRICT`, comportamiento que aborta la operación si existen referencias.
- Las restricciones de dominio se traducen en `CHECK`: precio estrictamente positivo, stock mínimo y máximo no negativos y `stock_minimo <= stock_maximo`.

DDL en MySQL 8+

```

CREATE DATABASE IF NOT EXISTS practicola_mysql;
USE practicola_mysql;

CREATE TABLE cliente (
    nro_cliente INT PRIMARY KEY,
    apellido VARCHAR(60) NOT NULL,
    nombre VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    CONSTRAINT uq_cliente_telefono UNIQUE (telefono)
);

CREATE TABLE producto (
    cod_producto INT PRIMARY KEY,
    descripcion VARCHAR(120) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock_minimo INT NOT NULL,
    stock_maximo INT NOT NULL,
    cantidad INT NOT NULL,
    CONSTRAINT ck_producto_precio CHECK (precio > 0),
    CONSTRAINT ck_producto_stock_rango CHECK (stock_minimo >= 0 AND stock_maximo >= 0),
    CONSTRAINT ck_producto_stock_logico CHECK (stock_minimo <= stock_maximo),
    CONSTRAINT ck_producto_cantidad CHECK (cantidad >= 0)
);

CREATE TABLE factura (
    nro_factura INT PRIMARY KEY,
    nro_cliente INT NOT NULL,
    fecha DATE NOT NULL,
    monto DECIMAL(12,2) NOT NULL,
    CONSTRAINT ck_factura_monto CHECK (monto > 0),
    CONSTRAINT fk_factura_cliente
        FOREIGN KEY (nro_cliente)
        REFERENCES cliente(nro_cliente)
        ON DELETE CASCADE
);

CREATE TABLE item_factura (
    cod_producto INT NOT NULL,
    nro_factura INT NOT NULL,
    cantidad INT NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_item_factura PRIMARY KEY (cod_producto, nro_factura),
    CONSTRAINT ck_item_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_item_precio CHECK (precio > 0),
    CONSTRAINT fk_item_producto
        FOREIGN KEY (cod_producto)
        REFERENCES producto(cod_producto)
        ON DELETE RESTRICT,
    CONSTRAINT fk_item_factura
        FOREIGN KEY (nro_factura)
        REFERENCES factura(nro_factura)
        ON DELETE CASCADE
);

```

DDL en PostgreSQL 14+

```

CREATE SCHEMA IF NOT EXISTS practicola;
SET search_path TO practicola;

CREATE TABLE cliente (
    nro_cliente INTEGER PRIMARY KEY,

```

```

    apellido VARCHAR(60) NOT NULL,
    nombre VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    CONSTRAINT uq_cliente_telefono UNIQUE (telefono)
);

CREATE TABLE producto (
    cod_producto INTEGER PRIMARY KEY,
    descripcion VARCHAR(120) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock_minimo INTEGER NOT NULL,
    stock_maximo INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    CONSTRAINT ck_producto_precio CHECK (precio > 0),
    CONSTRAINT ck_producto_stock_rango CHECK (stock_minimo >= 0 AND stock_maximo >= 0),
    CONSTRAINT ck_producto_stock_logico CHECK (stock_minimo <= stock_maximo),
    CONSTRAINT ck_producto_cantidad CHECK (cantidad >= 0)
);

CREATE TABLE factura (
    nro_factura INTEGER PRIMARY KEY,
    nro_cliente INTEGER NOT NULL,
    fecha DATE NOT NULL,
    monto NUMERIC(12,2) NOT NULL,
    CONSTRAINT ck_factura_monto CHECK (monto > 0),
    CONSTRAINT fk_factura_cliente
        FOREIGN KEY (nro_cliente)
        REFERENCES cliente(nro_cliente)
        ON DELETE CASCADE
);

CREATE TABLE item_factura (
    cod_producto INTEGER NOT NULL,
    nro_factura INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    CONSTRAINT pk_item_factura PRIMARY KEY (cod_producto, nro_factura),
    CONSTRAINT ck_item_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_item_precio CHECK (precio > 0),
    CONSTRAINT fk_item_producto
        FOREIGN KEY (cod_producto)
        REFERENCES producto(cod_producto)
        ON DELETE RESTRICT,
    CONSTRAINT fk_item_factura
        FOREIGN KEY (nro_factura)
        REFERENCES factura(nro_factura)
        ON DELETE CASCADE
);

```

2.2. Inciso b) Vehículos, personas, parquímetros y estacionamiento

Decisiones de modelado

- `vehiculo.patente`, `persona.dni` y `parquimetro.id_parquimetro` son las claves primarias del problema.
- La relación N:N entre vehículos y personas (un vehículo puede tener varios dueños) se resuelve con `duenio` como tabla intermedia, usando (`patente`, `dni`) como clave primaria compuesta.
- Se agrega `UNIQUE (calle, altura)` en `parquimetro`, dado que no tiene sentido que dos

parquímetros distintos compartan exactamente la misma ubicación.

- El dominio `nombre_y_apellido VARCHAR(45)` se emula a nivel de columna, ya que ni MySQL ni Oracle implementan `CREATE DOMAIN` del estándar.
- El dominio del atributo `color` queda restringido a `{gris, negro, azul}`: en MySQL se usa `ENUM` y en Oracle un `CHECK` con `IN (...)`.
- Para que no se pueda borrar un vehículo que ya fue estacionado, la FK desde `estacionamiento` hacia `vehiculo` se deja con el comportamiento por defecto (`RESTRICT/NO ACTION`).
- El `id_estacionamiento` es autonumerado: `AUTO_INCREMENT` en MySQL y la combinación `SEQUENCE + trigger BEFORE INSERT` en Oracle 10g/11g XE.

DDL en MySQL 8+

```
CREATE DATABASE IF NOT EXISTS practico1b_mysql;
USE practico1b_mysql;

CREATE TABLE persona (
    dni BIGINT PRIMARY KEY,
    nombre_y_apellido VARCHAR(45) NOT NULL,
    direccion VARCHAR(120) NOT NULL
);

CREATE TABLE vehiculo (
    patente VARCHAR(10) PRIMARY KEY,
    marca VARCHAR(40) NOT NULL,
    modelo INT NOT NULL,
    color ENUM('gris', 'negro', 'azul') NOT NULL,
    saldo_actual DECIMAL(10,2) NOT NULL DEFAULT 0,
    CONSTRAINT ck_vehiculo_saldo CHECK (saldo_actual >= 0)
);

CREATE TABLE parquemetro (
    id_parquemetro INT PRIMARY KEY,
    calle VARCHAR(80) NOT NULL,
    altura INT NOT NULL,
    CONSTRAINT uq_parquemetro_ubicacion UNIQUE (calle, altura),
    CONSTRAINT ck_parquemetro_altura CHECK (altura BETWEEN 0 AND 5000)
);

CREATE TABLE duenio (
    patente VARCHAR(10) NOT NULL,
    dni BIGINT NOT NULL,
    CONSTRAINT pk_duenio PRIMARY KEY (patente, dni),
    CONSTRAINT fk_duenio_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_duenio_persona
        FOREIGN KEY (dni)
        REFERENCES persona(dni)
        ON DELETE CASCADE
);

CREATE TABLE estacionamiento (
    id_estacionamiento INT AUTO_INCREMENT PRIMARY KEY,
    patente VARCHAR(10) NOT NULL,
    id_parquemetro INT NOT NULL,
    fecha DATE NOT NULL,
    saldo_inicio DECIMAL(10,2) NOT NULL,
    saldo_final DECIMAL(10,2) NOT NULL,
    hora_entrada TIME NOT NULL,
    hora_salida TIME,
```

```

CONSTRAINT ck_estacionamiento_saldos CHECK (saldo_inicio >= 0 AND saldo_final >= 0),
CONSTRAINT ck_estacionamiento_consumo CHECK (saldo_final <= saldo_inicio),
CONSTRAINT ck_estacionamiento_horas CHECK (hora_salida IS NULL OR hora_salida >=
hora_entrada),
CONSTRAINT fk_estacionamiento_vehiculo
    FOREIGN KEY (patente)
    REFERENCES vehiculo(patente),
CONSTRAINT fk_estacionamiento_parquimetro
    FOREIGN KEY (id_parquimetro)
    REFERENCES parquimetro(id_parquimetro)
);

```

DDL en Oracle XE 10g/11g

En Oracle no existe el concepto de “base de datos” como en MySQL o PostgreSQL: el equivalente práctico es crear un usuario que oficie de esquema dueño de las tablas. Por eso, el primer paso es conectarse como administrador y crear el usuario:

```

CREATE USER practico1b IDENTIFIED BY practico1b;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE TRIGGER TO practico1b;
ALTER USER practico1b QUOTA UNLIMITED ON USERS;

```

Una vez conectado como practico1b:

```

CREATE TABLE persona (
    dni NUMBER(10) PRIMARY KEY,
    nombre_y_apellido VARCHAR2(45 CHAR) NOT NULL,
    direccion VARCHAR2(120 CHAR) NOT NULL
);

CREATE TABLE vehiculo (
    patente VARCHAR2(10 CHAR) PRIMARY KEY,
    marca VARCHAR2(40 CHAR) NOT NULL,
    modelo NUMBER(4) NOT NULL,
    color VARCHAR2(10 CHAR) NOT NULL,
    saldo_actual NUMBER(10,2) DEFAULT 0 NOT NULL,
    CONSTRAINT ck_vehiculo_color CHECK (LOWER(color) IN ('gris', 'negro', 'azul')),
    CONSTRAINT ck_vehiculo_saldo CHECK (saldo_actual >= 0)
);

CREATE TABLE parquimetro (
    id_parquimetro NUMBER(10) PRIMARY KEY,
    calle VARCHAR2(80 CHAR) NOT NULL,
    altura NUMBER(4) NOT NULL,
    CONSTRAINT uq_parquimetro_ubicacion UNIQUE (calle, altura),
    CONSTRAINT ck_parquimetro_altura CHECK (altura BETWEEN 0 AND 5000)
);

CREATE TABLE duenio (
    patente VARCHAR2(10 CHAR) NOT NULL,
    dni NUMBER(10) NOT NULL,
    CONSTRAINT pk_duenio PRIMARY KEY (patente, dni),
    CONSTRAINT fk_duenio_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_duenio_persona
        FOREIGN KEY (dni)
        REFERENCES persona(dni)
        ON DELETE CASCADE
);

```

```

CREATE TABLE estacionamiento (
    id_estacionamiento NUMBER(10) PRIMARY KEY,
    patente VARCHAR2(10 CHAR) NOT NULL,
    id_parquimetro NUMBER(10) NOT NULL,
    fecha DATE NOT NULL,
    saldo_inicio NUMBER(10,2) NOT NULL,
    saldo_final NUMBER(10,2) NOT NULL,
    hora_entrada TIMESTAMP NOT NULL,
    hora_salida TIMESTAMP,
    CONSTRAINT ck_estacionamiento_saldos CHECK (saldo_inicio >= 0 AND saldo_final >= 0),
    CONSTRAINT ck_estacionamiento_consumo CHECK (saldo_final <= saldo_inicio),
    CONSTRAINT ck_estacionamiento_horas CHECK (hora_salida IS NULL OR hora_salida >=
    hora_entrada),
    CONSTRAINT fk_estacionamiento_vehiculo
        FOREIGN KEY (patente)
        REFERENCES vehiculo(patente),
    CONSTRAINT fk_estacionamiento_parquimetro
        FOREIGN KEY (id_parquimetro)
        REFERENCES parquimetro(id_parquimetro)
);

CREATE SEQUENCE seq_estacionamiento START WITH 1 INCREMENT BY 1 NOCACHE;

CREATE OR REPLACE TRIGGER trg_estacionamiento_bi
BEFORE INSERT ON estacionamiento
FOR EACH ROW
WHEN (NEW.id_estacionamiento IS NULL)
BEGIN
    SELECT seq_estacionamiento.NEXTVAL
    INTO :NEW.id_estacionamiento
    FROM dual;
END;
/

```

2.3. Inciso c) Clientes, automóviles, talleres y accidentes

Decisiones de modelado

- cliente.dni, automovil.patente, categoria.nro_categoria, taller.nro_taller y accidente.nro_accidente son las claves primarias.
- Aunque el enunciado solo declara explícitamente las FK de accidente, también se incluyen automovil.dni → cliente(dni) y automovil.nro_categoria → categoria(nro_categoria) porque cierran la integridad referencial del modelo.
- Para cumplir con el pedido de eliminar la información de los accidentes asociados al borrar un cliente, se propaga la cascada en automovil.dni y accidente.dni. Así, un único DELETE sobre cliente elimina todo lo relacionado.
- Las restricciones de dominio se traducen en CHECK: modelo BETWEEN 1990 AND 2015, marca IN ('FIAT', 'RENAULT', 'FORD') y tarifa >0 AND tarifa <10000.

DDL en PostgreSQL 14+

```

CREATE SCHEMA IF NOT EXISTS practico1c;
SET search_path TO practico1c;

CREATE TABLE cliente (
    dni BIGINT PRIMARY KEY,
    nombre VARCHAR(60) NOT NULL,

```



```

    apellido VARCHAR(60) NOT NULL,
    direccion VARCHAR(120) NOT NULL,
    tarifa INTEGER NOT NULL,
    CONSTRAINT ck_cliente_tarifa CHECK (tarifa > 0 AND tarifa < 10000)
);

CREATE TABLE categoria (
    nro_categoria INTEGER PRIMARY KEY,
    tasa NUMERIC(6,2) NOT NULL,
    CONSTRAINT ck_categoria_tasa CHECK (tasa > 0)
);

CREATE TABLE taller (
    nro_taller INTEGER PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    direccion VARCHAR(120) NOT NULL
);

CREATE TABLE automovil (
    patente VARCHAR(10) PRIMARY KEY,
    marca VARCHAR(10) NOT NULL,
    modelo INTEGER NOT NULL,
    dni BIGINT NOT NULL,
    nro_categoria INTEGER NOT NULL,
    CONSTRAINT ck_automovil_modelo CHECK (modelo BETWEEN 1990 AND 2015),
    CONSTRAINT ck_automovil_marca CHECK (marca IN ('FIAT', 'RENAULT', 'FORD')),
    CONSTRAINT fk_automovil_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_automovil_categoria
        FOREIGN KEY (nro_categoria)
        REFERENCES categoria(nro_categoria)
);

CREATE TABLE accidente (
    nro_accidente INTEGER PRIMARY KEY,
    dni BIGINT NOT NULL,
    patente VARCHAR(10) NOT NULL,
    nro_taller INTEGER NOT NULL,
    fecha DATE NOT NULL,
    costo NUMERIC(12,2) NOT NULL,
    CONSTRAINT ck_accidente_costo CHECK (costo > 0),
    CONSTRAINT fk_accidente_cliente
        FOREIGN KEY (dni)
        REFERENCES cliente(dni)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_automovil
        FOREIGN KEY (patente)
        REFERENCES automovil(patente)
        ON DELETE CASCADE,
    CONSTRAINT fk_accidente_taller
        FOREIGN KEY (nro_taller)
        REFERENCES taller(nro_taller)
);

```

DDL en Oracle XE 10g/11g

```

CREATE USER practico1c IDENTIFIED BY practico1c;
GRANT CREATE SESSION, CREATE TABLE TO practico1c;
ALTER USER practico1c QUOTA UNLIMITED ON USERS;

```

Y luego, ya como practico1c, se ejecuta el DDL:

```
CREATE TABLE cliente (  
    dni NUMBER(10) PRIMARY KEY,  
    nombre VARCHAR2(60 CHAR) NOT NULL,  
    apellido VARCHAR2(60 CHAR) NOT NULL,  
    direccion VARCHAR2(120 CHAR) NOT NULL,  
    tarifa NUMBER(5) NOT NULL,  
    CONSTRAINT ck_cliente_tarifa CHECK (tarifa > 0 AND tarifa < 10000)  
);  
  
CREATE TABLE categoria (  
    nro_categoria NUMBER(10) PRIMARY KEY,  
    tasa NUMBER(6,2) NOT NULL,  
    CONSTRAINT ck_categoria_tasa CHECK (tasa > 0)  
);  
  
CREATE TABLE taller (  
    nro_taller NUMBER(10) PRIMARY KEY,  
    nombre VARCHAR2(80 CHAR) NOT NULL,  
    direccion VARCHAR2(120 CHAR) NOT NULL  
);  
  
CREATE TABLE automovil (  
    patente VARCHAR2(10 CHAR) PRIMARY KEY,  
    marca VARCHAR2(10 CHAR) NOT NULL,  
    modelo NUMBER(4) NOT NULL,  
    dni NUMBER(10) NOT NULL,  
    nro_categoria NUMBER(10) NOT NULL,  
    CONSTRAINT ck_automovil_modelo CHECK (modelo BETWEEN 1990 AND 2015),  
    CONSTRAINT ck_automovil_marca CHECK (marca IN ('FIAT', 'RENAULT', 'FORD')),  
    CONSTRAINT fk_automovil_cliente  
        FOREIGN KEY (dni)  
        REFERENCES cliente(dni)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_automovil_categoria  
        FOREIGN KEY (nro_categoria)  
        REFERENCES categoria(nro_categoria)  
);  
  
CREATE TABLE accidente (  
    nro_accidente NUMBER(10) PRIMARY KEY,  
    dni NUMBER(10) NOT NULL,  
    patente VARCHAR2(10 CHAR) NOT NULL,  
    nro_taller NUMBER(10) NOT NULL,  
    fecha DATE NOT NULL,  
    costo NUMBER(12,2) NOT NULL,  
    CONSTRAINT ck_accidente_costo CHECK (costo > 0),  
    CONSTRAINT fk_accidente_cliente  
        FOREIGN KEY (dni)  
        REFERENCES cliente(dni)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_accidente_automovil  
        FOREIGN KEY (patente)  
        REFERENCES automovil(patente)  
        ON DELETE CASCADE,  
    CONSTRAINT fk_accidente_taller  
        FOREIGN KEY (nro_taller)  
        REFERENCES taller(nro_taller)  
);
```

3. Ejercicio 2: Carga de datos

Las inserciones se eligen de manera tal que cada consulta del Ejercicio 3 devuelva un resultado significativo y que las restricciones definidas (claves, dominios, integridad referencial) realmente entren en juego. Concretamente, en el inciso a) se incluye un cliente sin facturas, en el inciso b) se reparten estacionamientos sobre el parquímetro 9 y en el inciso c) se carga un cliente con cuatro accidentes.

3.1. Inciso a) – MySQL/PostgreSQL

```
INSERT INTO cliente (nro_cliente, apellido, nombre, direccion, telefono) VALUES
(1, 'Perez', 'Andres', 'San Martin 123', '3584010001'),
(2, 'Diaz', 'Bruno', 'Belgrano 450', '3584010002'),
(3, 'Gomez', 'Carlos', 'Mitre 980', '3584010003'),
(4, 'Luna', 'Diego', 'Constitucion 75', '3584010004');

INSERT INTO producto (cod_producto, descripcion, precio, stock_minimo, stock_maximo,
cantidad) VALUES
(101, 'Teclado mecanico', 15000.00, 5, 30, 12),
(102, 'Mouse optico', 8000.00, 10, 40, 25),
(103, 'Monitor 24', 90000.00, 2, 15, 6),
(104, 'Notebook 14', 450000.00, 1, 8, 3);

INSERT INTO factura (nro_factura, nro_cliente, fecha, monto) VALUES
(1001, 1, '2024-08-01', 106000.00),
(1002, 1, '2024-08-10', 450000.00),
(1003, 2, '2024-08-12', 46000.00),
(1004, 3, '2024-08-15', 98000.00);

INSERT INTO item_factura (cod_producto, nro_factura, cantidad, precio) VALUES
(103, 1001, 1, 90000.00),
(102, 1001, 2, 8000.00),
(104, 1002, 1, 450000.00),
(101, 1003, 2, 15000.00),
(102, 1003, 2, 8000.00),
(103, 1004, 1, 90000.00),
(102, 1004, 1, 8000.00);
```

Qué valida esta carga

- Diego Luna queda intencionalmente sin facturas, así aparece en la consulta de clientes sin ventas.
- Andrés Pérez tiene dos facturas con montos distintos, lo que permite ver con claridad el MAX y el MIN por cliente.
- Los montos declarados en cada factura coinciden con la suma de los precios y cantidades de sus ítems.

3.2. Inciso b) – MySQL

```
INSERT INTO persona (dni, nombre_y_apellido, direccion) VALUES
(30111222, 'Juan Perez', 'Laprida 120'),
(28999111, 'Mario Lopez', 'Lavalle 455'),
(33444555, 'Pedro Suarez', 'Mitre 900');

INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual) VALUES
('AA123BB', 'Renault', 2018, 'gris', 1500.00),
('AC456DD', 'Ford', 2020, 'negro', 800.00),
('AD789EE', 'Fiat', 2016, 'azul', 1200.00);
```

```

INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES
(9, 'Italia', 1200),
(10, 'Sarmiento', 845),
(11, 'San Martin', 2300);

INSERT INTO duenio (patente, dni) VALUES
('AA123BB', 30111222),
('AC456DD', 28999111),
('AD789EE', 33444555);

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES
('AA123BB', 9, '2024-08-02', 1500.00, 1350.00, '08:00:00', '09:30:00'),
('AC456DD', 9, '2024-08-02', 800.00, 650.00, '10:15:00', '11:45:00'),
('AD789EE', 10, '2024-08-03', 1200.00, 1000.00, '18:00:00', '19:40:00'),
('AA123BB', 11, '2024-08-04', 1350.00, 1250.00, '16:00:00', '16:50:00');

```

3.3. Inciso b) – Oracle

En Oracle se omite la columna `id_estacionamiento` en el `INSERT`, ya que el trigger `trg_estacionamiento_bi` se encarga de tomar el siguiente valor de la secuencia y completar el campo.

```

INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (30111222, 'Juan Perez', 'Laprida 120');
INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (28999111, 'Mario Lopez', 'Lavalley 455');
INSERT INTO persona (dni, nombre_y_apellido, direccion)
VALUES (33444555, 'Pedro Suarez', 'Mitre 900');

INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AA123BB', 'Renault', 2018, 'gris', 1500.00);
INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AC456DD', 'Ford', 2020, 'negro', 800.00);
INSERT INTO vehiculo (patente, marca, modelo, color, saldo_actual)
VALUES ('AD789EE', 'Fiat', 2016, 'azul', 1200.00);

INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (9, 'Italia', 1200);
INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (10, 'Sarmiento', 845);
INSERT INTO parquemetro (id_parquemetro, calle, altura) VALUES (11, 'San Martin', 2300);

INSERT INTO duenio (patente, dni) VALUES ('AA123BB', 30111222);
INSERT INTO duenio (patente, dni) VALUES ('AC456DD', 28999111);
INSERT INTO duenio (patente, dni) VALUES ('AD789EE', 33444555);

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AA123BB', 9, DATE '2024-08-02', 1500.00, 1350.00,
        TIMESTAMP '2024-08-02 08:00:00', TIMESTAMP '2024-08-02 09:30:00');

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AC456DD', 9, DATE '2024-08-02', 800.00, 650.00,
        TIMESTAMP '2024-08-02 10:15:00', TIMESTAMP '2024-08-02 11:45:00');

INSERT INTO estacionamiento
(patente, id_parquemetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AD789EE', 10, DATE '2024-08-03', 1200.00, 1000.00,
        TIMESTAMP '2024-08-03 18:00:00', TIMESTAMP '2024-08-03 19:40:00');

INSERT INTO estacionamiento

```

```
(patente, id_parquimetro, fecha, saldo_inicio, saldo_final, hora_entrada, hora_salida)
VALUES ('AA123BB', 11, DATE '2024-08-04', 1350.00, 1250.00,
        TIMESTAMP '2024-08-04 16:00:00', TIMESTAMP '2024-08-04 16:50:00');
```

3.4. Inciso c) – PostgreSQL

```
INSERT INTO cliente (dni, nombre, apellido, direccion, tarifa) VALUES
(30111222, 'Lucas', 'Gomez', 'Colon 120', 3500),
(28999111, 'Martin', 'Sosa', 'Belgrano 455', 4200),
(33444555, 'Sergio', 'Perez', 'Rivadavia 900', 2800);

INSERT INTO categoria (nro_categoria, tasa) VALUES
(1, 0.80),
(2, 1.10),
(3, 1.35);

INSERT INTO taller (nro_taller, nombre, direccion) VALUES
(1, 'Taller Centro', 'Sobremonte 100'),
(2, 'Taller Sur', 'Ruta 8 km 601');

INSERT INTO automovil (patente, marca, modelo, dni, nro_categoria) VALUES
('AAA111', 'FIAT', 2010, 30111222, 1),
('BBB222', 'FORD', 2012, 30111222, 2),
('CCC333', 'RENAULT', 2015, 28999111, 1),
('DDD444', 'FIAT', 2008, 33444555, 3);

INSERT INTO accidente (nro_accidente, dni, patente, nro_taller, fecha, costo) VALUES
(5001, 30111222, 'AAA111', 1, '2024-01-10', 150000.00),
(5002, 30111222, 'BBB222', 2, '2024-03-12', 230000.00),
(5003, 30111222, 'AAA111', 1, '2024-06-01', 80000.00),
(5004, 30111222, 'BBB222', 1, '2024-07-20', 120000.00),
(5005, 28999111, 'CCC333', 2, '2024-05-05', 95000.00),
(5006, 33444555, 'DDD444', 1, '2024-08-09', 110000.00);
```

Qué valida esta carga

- Lucas Gómez aparece con cuatro accidentes, lo que asegura que la consulta del inciso c) del Ejercicio 3 (con `HAVING COUNT(*) >3`) devuelva al menos una fila.
- Todas las marcas y modelos respetan los dominios definidos en el DDL, por lo que ningún `INSERT` es rechazado por los `CHECK`.

4. Ejercicio 3: Consultas SQL y álgebra relacional

Este ejercicio se centra en las consultas DML del repaso: filtros con `WHERE`, ordenamientos con `ORDER BY`, agregaciones con `GROUP BY/HAVING` y subconsultas. Las cuatro primeras consultas son las pedidas por el enunciado y la última corresponde a la parte de álgebra relacional.

4.1. a) Clientes sin ventas (base del inciso 1.a)

El enunciado pide listar los clientes (con todos sus datos) que no tengan ninguna factura asociada, ordenando el resultado por apellido y nombre en forma descendente.

```
SELECT c.nro_cliente,
       c.apellido,
       c.nombre,
       c.direccion,
       c.telefono
```

```
FROM cliente c
WHERE NOT EXISTS (
    SELECT 1
    FROM factura f
    WHERE f.nro_cliente = c.nro_cliente
)
ORDER BY c.apellido DESC, c.nombre DESC;
```

Idea

La consulta se modela naturalmente con una subconsulta correlacionada: NOT EXISTS se lee literalmente como “no existe ninguna factura asociada a este cliente”. Es una formulación equivalente a un LEFT JOIN ... WHERE factura.nro_cliente IS NULL o a WHERE nro_cliente NOT IN (...), pero se comporta correctamente frente a posibles NULL y suele ser la opción más limpia. Sobre los datos del Ejercicio 2, el resultado esperado es Diego Luna, único cliente sin facturas.

4.2. b) Vehículos que usaron el parquímetro 9 (base del inciso 1.b)

El enunciado pide indicar modelo y color de los vehículos que utilizaron el parquímetro 9. Se incluye también la patente para identificar inequívocamente cada vehículo, ya que sin ella podrían aparecer filas que parezcan repetidas (mismo modelo y color, pero autos distintos).

```
SELECT DISTINCT v.patente,
               v.modelo,
               v.color
FROM vehiculo v
JOIN estacionamiento e
  ON e.patente = v.patente
WHERE e.id_parquimetro = 9;
```

Idea

La consulta cruza vehiculo con estacionamiento por la patente y filtra los registros del parquímetro 9. El DISTINCT es importante porque un mismo vehículo puede haberse estacionado allí en varias ocasiones; sin él, cada estacionamiento generaría una fila duplicada en el resultado. Con los datos cargados, las patentes AA123BB y AC456DD aparecen en el listado.

4.3. c) Clientes con más de 3 accidentes (base del inciso 1.c)

```
SELECT c.dni,
       c.nombre,
       c.apellido
FROM cliente c
JOIN accidente a
  ON a.dni = c.dni
GROUP BY c.dni, c.nombre, c.apellido
HAVING COUNT(*) > 3;
```

Idea

Es un caso típico de agregación con filtro posterior. GROUP BY arma un grupo por cada cliente y HAVING COUNT(*) >3 deja solo aquellos cuyos accidentes superan los tres. La diferencia clave con WHERE es que HAVING se aplica sobre filas ya agrupadas, que es exactamente lo que se necesita aquí. Con la carga del Ejercicio 2, el único cliente que cumple es Lucas Gómez, que registra cuatro accidentes.

4.4. d) Máximo y mínimo monto de factura por cliente (base del inciso 1.a)

```
SELECT c.nro_cliente,
       c.nombre,
       c.apellido,
       MAX(f.monto) AS monto_maximo,
       MIN(f.monto) AS monto_minimo
FROM cliente c
JOIN factura f
  ON f.nro_cliente = c.nro_cliente
GROUP BY c.nro_cliente, c.nombre, c.apellido
ORDER BY c.nro_cliente;
```

Idea

Las funciones MAX y MIN aplicadas sobre `monto` resuelven directamente el pedido. Como la consulta utiliza un join interno, solo aparecen los clientes con al menos una factura. Esto es deseable: un cliente sin facturas no tiene un máximo ni un mínimo bien definido y, además, su caso ya se cubre en el inciso a). El `ORDER BY c.nro_cliente` es un detalle de presentación para que el listado quede ordenado de forma estable.

4.5. e) Tres consultas en álgebra relacional

Para esta parte se elige como dominio la base del inciso 1.a, porque sus relaciones son simples y permiten ilustrar con claridad cada operador. Entre las tres consultas se cubren los operadores que pide el enunciado: selección, proyección, unión, intersección y producto cartesiano.

Consulta 1 — Productos con stock por debajo o igual al mínimo

Esta consulta detecta productos que necesitan reposición. Se utiliza una unión de dos selecciones para ejemplificar el operador $\cup$, aunque podría escribirse con un único `WHERE cantidad <= stock_minimo`.

Álgebra relacional:

```
pi_{cod_producto, descripcion}(sigma_{cantidad < stock_minimo}(Producto))
union
pi_{cod_producto, descripcion}(sigma_{cantidad = stock_minimo}(Producto))
```

SQL equivalente:

```
SELECT cod_producto, descripcion
FROM producto
WHERE cantidad < stock_minimo

UNION

SELECT cod_producto, descripcion
FROM producto
WHERE cantidad = stock_minimo;
```

Operadores usados: selección, proyección y unión.

Consulta 2 — Productos facturados por debajo del stock máximo

La consulta combina la información del registro de ventas (`ItemFactura`) con la del catálogo (`Producto`) para identificar productos que ya tuvieron movimiento pero todavía conservan

capacidad de stock.

Álgebra relacional:

```
pi_{cod_producto}(ItemFactura)
interseccion
pi_{cod_producto}(\sigma_{cantidad < stock_maximo}(Producto))
```

SQL equivalente:

```
SELECT DISTINCT i.cod_producto
FROM item_factura i
JOIN producto p
  ON p.cod_producto = i.cod_producto
WHERE p.cantidad < p.stock_maximo;
```

Operadores usados: selección, proyección e intersección. En SQL se utiliza un JOIN con DISTINCT porque modela exactamente la intersección sobre el conjunto de códigos de producto que cumplen ambas condiciones.

Consulta 3 — Pares cliente-factura a partir del producto cartesiano

Esta consulta sirve para ilustrar el operador de producto cartesiano. El producto entre **Cliente** y **Factura** genera todas las combinaciones posibles, y la selección posterior se queda solo con los pares que coinciden por **nro_cliente**. Esa combinación de operadores es, conceptualmente, lo que un join interno hace por debajo.

Álgebra relacional:

```
pi_{Cliente.nro_cliente, Cliente.apellido, Factura.nro_factura}
(
  \sigma_{Cliente.nro_cliente = Factura.nro_cliente}
  (Cliente x Factura)
)
```

SQL equivalente:

```
SELECT c.nro_cliente,
       c.apellido,
       f.nro_factura
FROM cliente c
JOIN factura f
  ON c.nro_cliente = f.nro_cliente;
```

Operadores usados: producto cartesiano, selección y proyección.

5. Conclusión

El práctico permitió repasar de forma integral las construcciones más usadas de SQL: definición de tablas con sus restricciones, carga de datos coherente con el modelo y consultas que cubren desde filtros simples hasta agregaciones y subconsultas correlacionadas. También obligó a confrontar diferencias prácticas entre motores, en particular cómo emular dominios cuando el motor no soporta CREATE DOMAIN y cómo implementar autonumeración en Oracle 10g/11g XE mediante secuencias y triggers.

Las decisiones de modelado se justificaron a partir del enunciado, manteniendo consistencia entre los tres incisos y respetando las reglas de borrado pedidas. La carga de datos se diseñó pensando

en las consultas posteriores, lo que facilitó verificar que las restricciones se aplican correctamente y que las consultas devuelven resultados representativos. La parte de álgebra relacional permitió pensar las consultas en términos de operadores formales y traducirlas a SQL, recuperando la conexión entre el modelo teórico y la práctica.